Unit - V Basics of Network Programming

5.1 Socket Overview: Client/Server , reserved Sockets , proxy servers , Internet Addressing
5.2 Java and the Net: The networking classes and interfaces, InetAddress : Factory Methods , Instance Methods
5.3 TCP/IP Client and Server Sockets, datagram sockets, datagram packets
5.4 The URL Class, URLConnection class

Networking Basics:-

- When two or more computers need to communicate with each other to share data or resources,then the communication between them can take place through networking.
- Networking enables sharing of resources and communication.
- Internet is a network of networks.
- **Overview of Socket:-**
- A Socket is a handle to a communication link with another application over the network.
- In other words,a socket is a software interface that connects an application to the network.
- Sockets are often used in client/server applications.

- A socket on one computer can “talk” talk to a socket on another computer via this a communication channel
- Two main types of sockets supported in Java
  - TCP sockets (implemented by the Socket class)
  - UDP sockets (implemented by the DatagramSocket class)

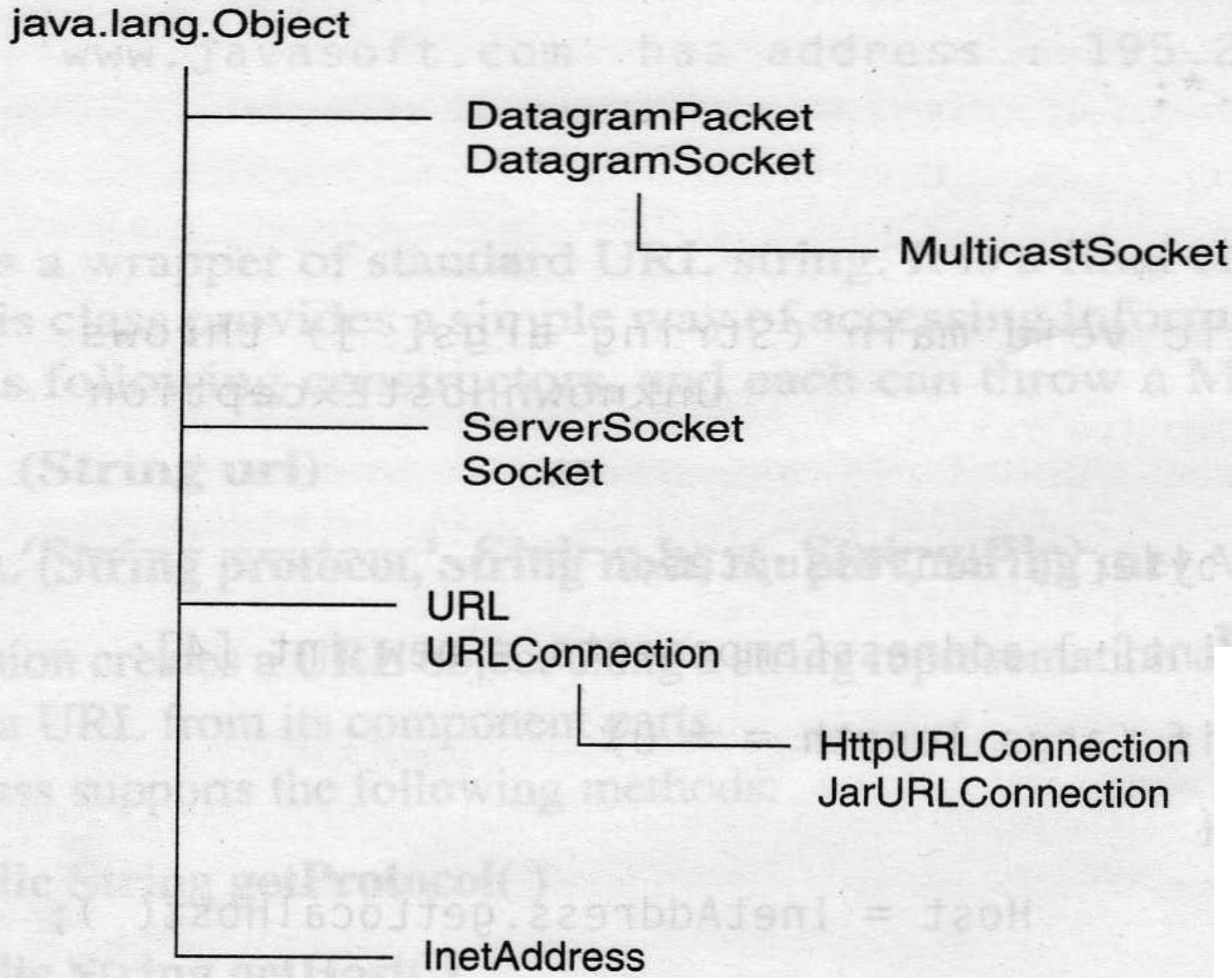
- **Client-Server Architecture:-**
- A computer,which requests for some service from another computer is called a client.
- The one that processes the request is called as server.
- A server waits till one of its clients makes a request.
- Socket allows a single computer to serve different types of information to different clients at the same time.
- This is managed by the port,which is a numbered socket on a particular machine.
- A server process can listen to a port number,although each session is unique.
- To manage multiple client connections,A server process must be multithreaded or multiplexing the simultaneous I/O.

- **Reserved Sockets:-**
- TCP/IP has 65,536 different TCP ports or sockets to which every machine can communicate.first 1024 ports are reserved for system’s predefined services.

Port Nam	Port Number	Description
echo	7	Displays whatever is sent by the user.
discard	9	Discards whatever is sent by the user.
daytime	13	Gives the local time of the destination machine.
qotd	17	Gives the "quote of the day" for the machine.
changer	19	Generates a test stream of characters.
ftp	21	FTP Port.
telnet	23	Telnet Protocol Port.
smtp	25	SMTP Port.
finger	79	Finger Protocol Port.
http	80	Web Server Port.
pop3	110	Pop Version 3 Port.
nntp	119	NNTP Port.

- **Protocols:-**
- Communication between computers in a network or a different network requires certain set of rules called protocols.
- Java networking is done using TCP/IP protocol.
- Some of the different kinds of protocol available are
- HTTP:-(Hyper Text Transfer Protocol)enables interaction with the internet.
- FTP:-(File Transfer Protocol)enables transfer of files between computers.
- SMTP:-(Simple Mail Transfer Protocol)provides e-mail facility.
- NNTP:-(Network News Transfer Protocol) acts as a bulletin board for sharing news.

- **Proxy Server:-**
- A *proxy server connects* the client side of a protocol to another server.
- This is often required when clients have certain restrictions on which servers they can connect to.
- Thus, a client would connect to a proxy server, which did not have such restrictions, and the proxy server would in turn communicate for the client.
- A proxy server has the additional ability to filter certain requests or cache the results of those requests for future use.
- A caching proxy HTTP server can help reduce the bandwidth demands on a local network's connection to the Internet.
- The **purpose of a proxy server** is to act as an **intermediary** between a user (client) and the internet (server). Instead of connecting directly to a website, your request goes through the proxy server first.


java.lang.Object
DatagramPacket
DatagramSocket
MulticastSocket
ServerSocket
Socket
URL
URLConnection
HttpURLConnection
JarURLConnection
InetAddress

- **Internet Addressing:-**
- Every computer connected to a network has a unique IP address.
- An Internet address is a number that uniquely identifies each computer on the Net.
- An IP address is represented by 32-bit or 128-bit unsigned number.
- An instance of InetAddress represents the IP address with its corresponding host name.
- There are two types of address types:
- Unicast and Multicast.
- The Unicast is an identifier for a single interface whereas Multicast is an identifier for a set of interfaces.

- There are 32 bits in an IP address, which has sequence of four numbers between 0 and 255 separated by dots or periods(.).
- It is possible to connect to the Internet either directly or by using Internet Service Provider.
- By connecting directly to the Internet,the computer is assigned with a permanent IP address.
- In case connection is made using ISP,it assigns a temporary IP address for each session.
- Ex-80.0.0.53

- **Domain Naming Service:-**
- It is very difficult to remember a set of numbers (IP address)to connect to the Internet.
- The Domain Naming Service (DNS) is used to overcome this problem.
- It maps one particular IP address to a string of characters.
- The name of an Internet address,called its domain name,describes a machine's location in a name space from right to left.
- Ex:- www.yahoo.com
- Implies com is the domain name reserved for US commercial sites, yahoo is the name of the company and www is the name of the specific computer,which is yahoo sever.

InetAddress class:-

- The **`java.net.InetAddress`** class represents an IP address.
- It converts numeric addresses to host names and host names to numeric addresses.
- It is used by other network classes like **`Socket`** and **`ServerSocket`** to identify hosts
- The InetAddress class is used to encapsulate both the numerical IP address and the domain name for that address.
- There are no public **`InetAddress()`** constructors. To create an InetAddress object, you have to use one of the available factory methods.
- Factory methods are merely a convention whereby static methods in a class return an instance of that class.

- **Factory method:-**

a)static InetAddress getLocalHost( )throws UnknownHostException

- Returns the InetAddress object that represents the localhost.

b)static InetAddress getByName(String hostName) throws UnknownHostException

- Returns the InetAddress for a host name passed to it.if these methods are unable to resolve the host name,they throw an UnknownHostException.

c)static InetAddress[ ] getAllByName(String hostName) throws UnknownHostException

- Returns an array of InetAddress that represent all of the addresses that a particular name resolves to.if these methods are unable to resolve the host name,they throw an UnknownHostException.

- **Instance Methods:-**
- The InetAddress class also has several other methods.

  Boolean equals(Object other)
- Returns **true** if this object has the same Internet address as *other*.

  byte[ ] getAddress( )
- Returns a four-element byte array that represents the object's Internet address in network byte order.

  String getHostAddress( )
- Returns a string that represents the host address associated with the **InetAddress** object.

  String getHostName( )
- Returns a string that represents the host name associated with the **InetAddress** object.

boolean isMulticastAddress( )

- Returns **true** if this Internet address is a multicast address. Otherwise, it returns **false**.

String toString( )

- Returns a string that lists the host name and the IP address for convenience;
- for example, "starwave.com/192.147.170.6".

URL:-

- IP addresses uniquely identify systems on the Internet.
- TCP port and UDP port identify services on a system.
- URL provides a uniform format for identifying resources on the Internet.
- The general format of a URL is

scheme_name://host:port/file_info#internal_reference

Where **scheme_name** is the protocol used,such as,HTTP,FTP,Gopher etc.

host is the IP address or the domain name of the remote system,

Port is the port number of the service required,

file_info is the resource requested from the remote system,

Internal reference is the identifier of a named link within the HTML page.

Ex:-

http://www.who.org:8080/index.html

http://java.run.com/developers/welcome.html#footer

URL Class:-

- The URL class is a wrapper of standard URL string.it is a final class and is directly inherited from the Object class.
- This class provides a simple way of accessing information across the Internet using URLs.
- URL class has following constructors,and each can throw a **MalformedURLException.**

  URL(String url)

  URL(String protocol,String host,String file)

  URL(String protocol,String host,int port,String file)

  First version creates a URL object using a string representation of an absolute URL and next two versions create a URL from its component parts.

Method	Description
public String getProtocol()	it returns the protocol of the URL.
public String getHost()	it returns the host name of the URL.
public String getPort()	it returns the Port Number of the URL.
public String getFile()	it returns the file name of the URL.
public String getAuthority()	it returns the authority of the URL.
public String toString()	it returns the string representation of the URL.
public String getQuery()	it returns the query string of the URL.
public String getDefaultPort()	it returns the default port of the URL.

- These methods return various components of the URL object.The getPort() method returns -1 if the port is not specified in the invoking URL object.

  public String toExternalForm()

  public boolean sameFile(URL u)
- The toExternalForm() method returns a string representation of the URL object,and
- sameFile() method returns true if both URLs have same protocol,host,port and file name.

URLConnection class:-

- **URLConnection** is abstract class that represents connection between an application and URL and provides methods common to the subclass.
- Subclasses of the **URLConnection** class, namely **HttpURLConnection** and **JarURLConnection** support protocol specific connection to URL.
- Object of the URLConnection class created by invoking the **openConnection( )** method of the URL object.

  URL u = new URL("http", "www.java2s.com", 80, "/");

  URLConnection uCon = u.openConnection();
- The following methods are supported by URLConnecton class:-

public Object getContent( )
public URL getURL( )
public String getContentEncoding( )
public int getContentLength( )
public String getContentType( )
public long getdate( )
public long getLastModified( )

These methods returns information about various properties of the resource specified by the URL object that is associated with the invoking URLConnecton object.

Connection–oriented protocol:-

- **TCP/IP Sockets:-**
- TCP/IP sockets are used to implement reliable,bidirectional,persistent,point to point,stream based connection between two hosts on the internet.
- A socket can be used to connect java's Input/Output system to other programs,that may reside either on the local machine or any other machine or any other machine on the Internet.
- In java,TCP sockets are classified as:
- TCP Server Socket
- TCP Client Socket
- The **java.net** package provides two classes that allow you to create the two kinds of sockets.
- The classes are **Socket** and **ServerSocket** respectively.
- The **Socket** class,basically used to create client sockets,is designed to connect to server sockets and initiate protocol exchanges.
- The **ServerSocket** class,basically used to create server sockets,is designed to be a listener,which waits for clients to connect before doing anything.

Socket Class:-

- The **Socket** class encapsulates a two-way communication mechanism that is connected at both ends to an IP address and port number.
- It implements a stream-based,connection-oriented data communication.
- When you create a **Socket** object,it implicitly establishes the connection between the client and the server.
- The **Socket** class provides the following constructors to create client sockets:

  Socket(String hostname,int port) throws UnknownHostException,IOException

  Socket(InetAddress ipAddress,int port)throws IOException
- These constructors are used to create a socket by specifying hostname and port or by using an existing InetAddress object and a port.
- After you have created a socket,you can get the address and port details associated with the socket by the use of the following methods:

  InetAddress getInetAddress()

  Int getPort()

  Int getLocalPort()

- Since TCP is a stream based protocol,every TCP socket is associated with input and output streams.
- To return the **InputStream** and **OutputStream** objects,that can be used to read and send data by the socket,the following methods are used:

  InputStream getInputStream()

  OutputStream getOutputStream ()

ServerSocket class:-

- The ServerSocket class is used to create servers that listen for either local or remote client programs to connect to them on standard ports.
- The ServerSocket object can create a Socket object to receive and transmit data through the socket.
- This class has the following constructors:

  ServerSocket(int port) throws IOException

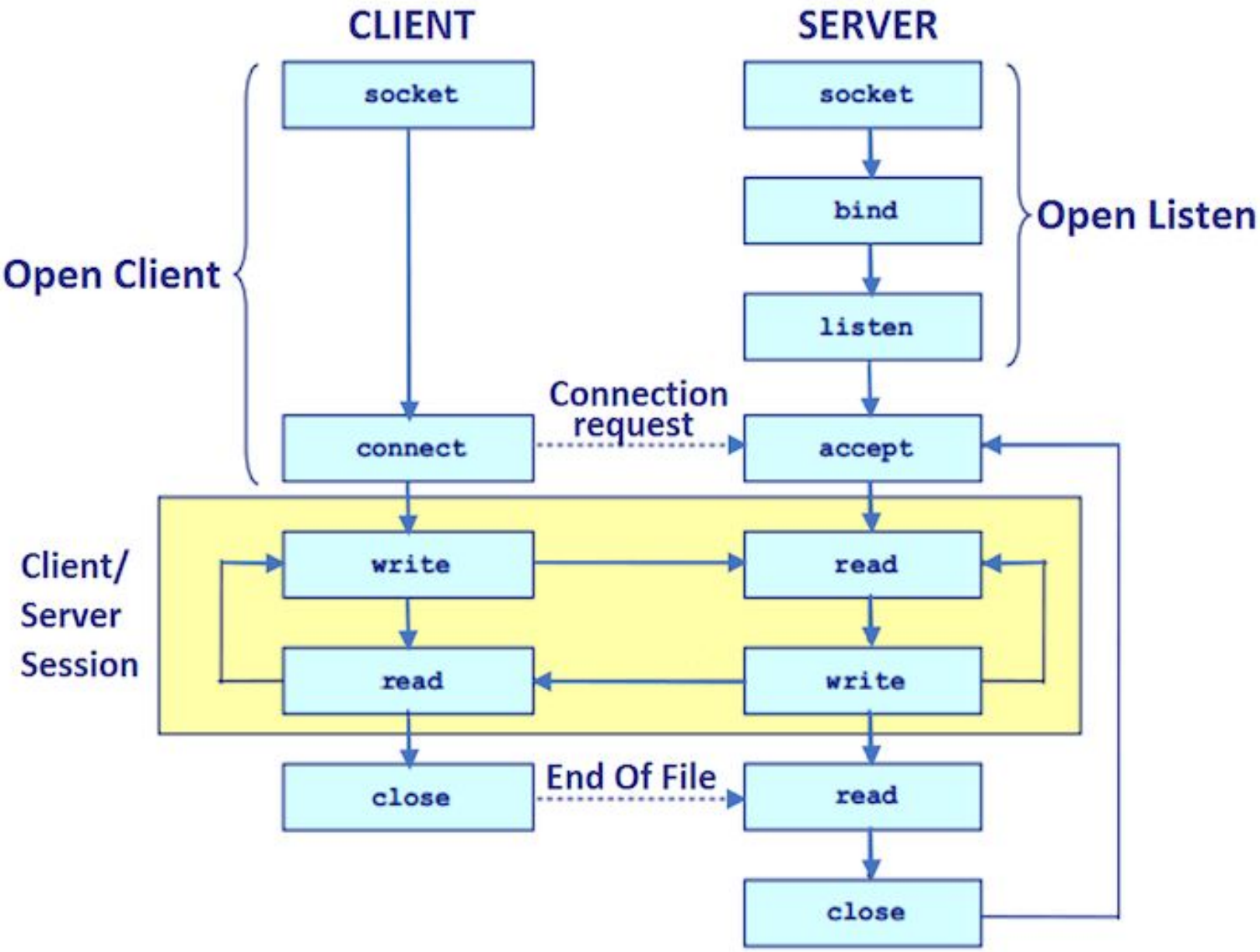
  ServerSocket(int port,int queuelength)throws IOException
- These constructors are used to create a server socket on the specified port with a default queue length of 50 or with the specified queue length.both can throw an IOException.
- This class provides the following methods:
- **Socket accept()**
- This method listens for a connection request and accepts the connection.when the connection is accepted,a Socket object is returned,that can be used to send and receive data through the connection.

InetAddress getInetAddress()

Int getLocalPort()

void close()

- These methods returns an IP address and port number used by the invoking ServerSocket object.
- Last method closes the socket and releases any system resources associated with it.


CLIENT
SERVER
socket
socket
bind
listen
Open Listen
Open Client
Connection request
connect
accept
Client/ Server Session
write
read
read
write
End Of File
close
read
close

- **To Write a Client program:**

Step 1:Create the socket for client using server's IP address and port number.

Step 2:create input and output streams for client socket.

Step 3:Conduct the conversation based on the agreed protocol.

Step 4:close the Stream.

Step 5:close the socket.

Writing Client/Server Systems using TCP:

- **To Write a Server program:**

Step 1:Create the server socket and begin listening.

Step 2:call the accept() method to get new connections as Socket object.

Step 3:create input and output streams for the returned socket.

Step 4:Conduct the conversation based on the agreed protocol.

Step 5:close the Stream and Socket.

Step 6:close the server socket.

UDP

- Unreliable Datagram Protocol
- Packet Oriented, not stream oriented like TCP/IP
- Much faster but no error correction
- Must fit data into packets of about 8K or less.

The UDP Classes

- Java's support for UDP is contained in two classes:

  `java.net.DatagramSocket`

  `java.net.DatagramPacket`

- A datagram socket is used to send and receive datagram packets.

DatagramPacket and DatagramSocket classes:-

- The java.net package provides two classes,namely**,DatagramPacket** and **DatagramSocket**,through which you can communicate with remote systems.
- A UDP Datagram is encapsulated in an instance of the class DatagramPacket.
- **DatagramPacket:**
- **DatagramPacket** is a message that can be sent or received. If you send multiple packet, it may arrive in any order. Additionally, packet delivery is not guaranteed.
- Commonly used Constructors of DatagramPacket class:
- **DatagramPacket(byte[] barr, int length):** it creates a datagram packet. This constructor is used to receive the packets.
- **DatagramPacket(byte[] barr, int length, InetAddress address, int port):** it creates a datagram packet. This constructor is used to send the packets.

- Various Method Defined in the DatagramPacket class are:

  InetAddress getAddress()

  Int getPort()

  Byte[ ] getData()

  Int getLength()
- These methods return the destination machine's address,the port number,data contained in the datagram,and the length of the valid data contained in the byte array returned from the **getData()** method.
- **DatagramPacket** class does not provide methods to send or receive data or datagrams.
- This job is taken up by the **DatagramSocket** class.

- **Java DatagramSocket class:**
- **Java DatagramSocket** class represents a connection-less socket for sending and receiving datagram packets.
- A datagram is basically an information but there is no guarantee of its content, arrival or arrival time.
- Commonly used Constructors of DatagramSocket class
- **DatagramSocket() throws SocketException:** it creates a datagram socket and binds it with the available Port Number on the localhost machine.
- **DatagramSocket(int port) throws SocketException:** it creates a datagram socket and binds it with the given Port Number.
- **DatagramSocket(int port, InetAddress address) throws SocketException:** it creates a datagram socket and binds it with the specified port number and host address.

- The important methods are:

  void send(DatagramPacket packet)throws IOException
- This method sends the datagram's data to the previously defined host and port by taking a DatagramPacket object as a parameter.

  void receive(DatagramPacket packet)throws IOException
- This method receives a datagram by taking a DatagramPacket object as a parameter.
- To extract data from the received datagram use the **getData()** method defined in the **DatagramPacket** class.

- **void close() :**This method closes the opened datagram socket.